

Autoencoder²

Felix Céard-Falkenberg

Julian van den Bergh

Jorryt de Jong

Filipe Nibbering

Abstract—

I. INTRODUCTION

- Paragraph about Neuroscience
- Paragraph about EEG/MEG
- Paragraph about Deep learning in Neuro Science
- Paragraph about Deep Learning for MEG
- What we are doing

II. RELATED WORK

III. METHODOLOGY

A. Data

We got given Magnetoencephalography (MEG) scans of the brains, for various persons performing one of four tasks: resting, a motor skill, a math story, and a working memory task. In this work, we will examine two cases: whenever the scans are made on a single individual performing different task, and whenever we have data from multiple individual performing the same task. We will denote the dataset for the first case as $\mathcal{D}_{\text{Intra}}$ and we will use $\mathcal{D}_{\text{Cross}}$ for the dataset consisting of multiple individuals.

Each MEG scan consists of 35 624 timesteps of 248 sensors, which are sampled at 2034 Hz. For $\mathcal{D}_{\text{Intra}}$, we have a 8 scans for each task in the train set, and a test set with 2 scans per task. The $\mathcal{D}_{\text{Cross}}$ dataset includes the data from $\mathcal{D}_{\text{Intra}}$ and therefore has 8 scans per task for each of the three individuals in the training set, and 4 scans in each of the three test sets, which consist of scans from only one individual.

We preprocess the data for $\mathcal{D}_{\text{Intra}}$ and $\mathcal{D}_{\text{Cross}}$ as follows: Because each scan consists of time series data (see Figure 1), we hypothesize that critical information can be extracted from transforming the data into the frequency domain. To this end, we compute the Mel Spectrogram [1] of each sensor, extracting 64 frequency bands from 0 Hz to 300 Hz, due to the importance of low frequencies in MEG data [2]. Before converting the sensor data to the frequency domain, we first normalize each sensor to be zero centered and have a maximum value of 1. To avoid data spillage, we pre-compute the maximum value and the average mean for each sensor in the train set, and use these values for both the train and test scans.

B. Model

Due to the limited number of scans compared to the tasks, generalizing to unseen participants is difficult, as each unseen individual will likely result in a distribution shift. To this end, we propose to first extract high-quality features for each sensor by using an auto encoder; by compressing the most important information of each sensor into a single vector, we hope to

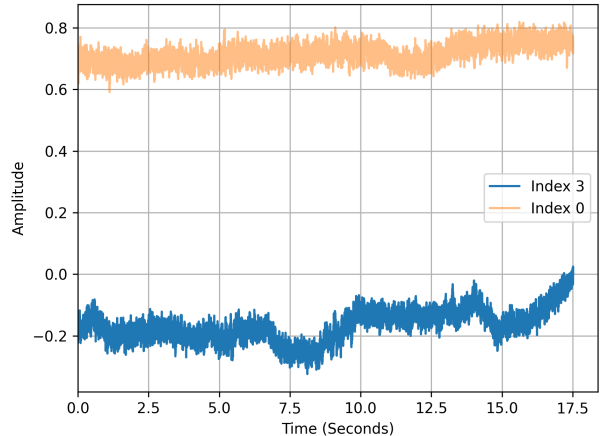


Fig. 1. Normalized data for 2 different sensors of the same scan from $\mathcal{D}_{\text{Intra}}$.

reduce the search space needed for a classifier to discriminate between the four tasks.

As each sensor consists of time-series data, we use time-series model to compress and reconstruct the data. An LSTM is a natural choice, due to its strong ability to train on long sequence of data. The encoder consists of a n LSTM layers with hidden dimensionality m . After encoding a sequence by the encoder, we pass the last layer's final hidden state to the decoder, which consists of n LSTM layers, followed by a linear layer to match the reconstructed size.

Because we use the LSTM for compressing the sequence of data, we investigate whether bidirectional processing improves the compression of the sequences.

C. Training

Because we do not train a variational autoencoder, we use the Mean Squared Error (MSE) loss to train the encoder and decoder end-to-end. We use the Adam optimizer with a base learning rate of 10^{-2} , which we reduce after each epoch using a cosine annealing scheduler until the learning rate reaches 10^{-6} . Recent advancements in training deep neural networks show that a non-fix learning rate benefits the optimization of neural networks [3]. A plot of the learning rate can be found in Figure 7 in the Appendix.

Due to the large amount of raw data (10GB), we are unable to perform cross validation, and therefore only report on the performance on the hold out set.

Finally, we trained models using 16 bit float points, as it speeds up the training significantly while not altering the

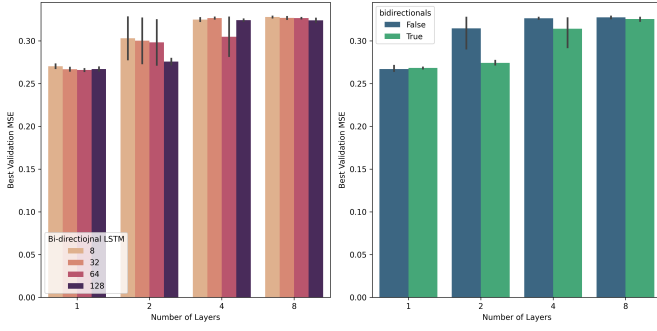


Fig. 2. Raw data for 2 sensors of the same scan from $\mathcal{D}_{\text{Intra}}$.

obtained models significantly. We trained all our models on a MacBookPro M4 pro chip with 48 GB RAM.

IV. RESULTS

A. Hyperparameter Tuning

Our LSTM autoencoder has multiple hyperparameter that we can tune, namely the number of layers for the LSTM, the hidden dimensionality, and whether we use a bidirectional LSTM for encoding the sequence. We run a grid search on the hyperparameter, which we report in Figure 2. It is easy to see that the best model validation loss is achieved by having the fewest LSTM layers (only one), a high hidden dimensionality (128), and bidirectional processing.

While the loss is not very informative on the training, we can verify the quality of the trained auto encoder by plotting the embeddings on a 2D plane. For each sensor of each scan of $\mathcal{D}_{\text{Intra}}$, we extract the last hidden state from the encoder and use T-SNE as a dimensionality reduction. We plot the embeddings in Figure 3.

We see that not only are the samples clustered around samples of similar tasks, often, similar sensors are also close together. This indicates that the structure of the latent representation is rich enough to capture the most important details about the sensor time series.

Correlation between raw distance and embedding distance

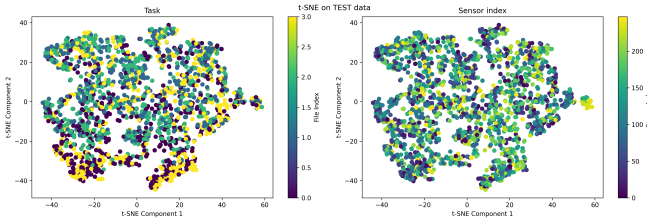


Fig. 3. T-SNE embedding for the LSTM autoencoder.

B. Predicting using the Embedding

We now investigate the discriminative power of the extracted features. To this end, we compute the embedding for each of the 248 features for each sample in $\mathcal{D}_{\text{Intra}}$. We then concatenate all embeddings together to form on large vector

for each MEG scan. Because the best performing model has a hidden dimensionality of 128, and because we are using a bidirectional LSTM, we have a embedding of 256 dimensions for each sensor of each scan. By concatenating all sensor embeddings into one vector, we obtain a vector with 63 488 dimensions.

Before fitting a machine learning model on this data, we aim to investigate how discriminative the concatenation of these features are. To this end, we use PCA to reduce the dimensionality of the embedding to 2 features, and plot these dimensionality reduced samples in Figure 4. Interestingly, without explicitly training the autoencoder to discriminate between the classes, by simply embedding the sensor signals, we are able to easily discriminate between each of the four classes, for both the train and validation set.

For completeness, we ran k-mean clustering on the concatenated labels of the train set, and obtain clusters that perfectly match the ground truth labels, for both the train and validation set. Therefore, by simply encoding and decoding the data, we are able to achieve 100% accuracy in predicting the state of the patient. We show in Figure 8 in the Appendix the predicted labels from k-means.

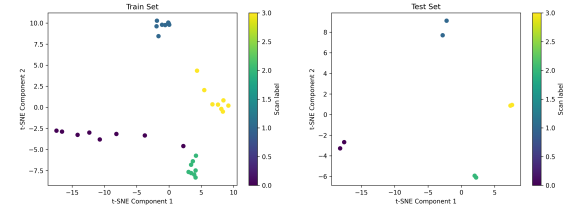


Fig. 4. PCA embedding for the LSTM autoencoder. Each dot represents a different scan.

Unseen data. We now investigate how well the learned embeddings generalizes to unseen patients. We use the same autoencoder previously trained, and the same k-means clustering for predicting the classes and obtain not only PCA embeddings that differ greatly from those in Figure 4, but also the k-means return one label for all samples. We show a plot of the embeddings in the Appendix in Figure 9.

This is not surprising, as the distribution shift between multiple patients can be very large in neuro science []. Therefore, we re-train our LSTM auto-encoder for compressing the individual sensor signals on $\mathcal{D}_{\text{Cross}}$, and use PCA on the whole $\mathcal{D}_{\text{Cross}}$ training set.

Our results follow the same trend as previously observed, namely that a larger hidden dimensionality and a bidirectional encoding achieves the lowest loss (see Figure 10 in the Appendix). However, contrary to the results on $\mathcal{D}_{\text{Intra}}$, the embeddings of the scans from the autoencoder on $\mathcal{D}_{\text{Cross}}$ are not well separated w.r.t. the ground truth labels. We plot the PCA and t-sne transformation of the embeddings of the autoencoder of each scan of $\mathcal{D}_{\text{Cross}}$ in Figure 5. We can see that, while the training samples are clustered together in the latent space, the embeddings do not generalize to new patients yet.

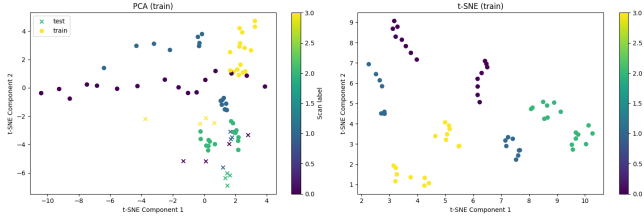


Fig. 5. Caption

Directly training a classifier, such as a logistic regression, on the raw embeddings does not result in good classification. We therefore aim to further separate the embeddings. To this end, we train a second autoencoder on the embeddings of the first autoencoder with a contrastive loss. This way, we not only obtain better plots by reducing the dimensionality of the embeddings, but by directly optimizing the separation of the scans w.r.t. their labels, training a classifier on those embeddings will result in better performance.

The architecture of the second encoder is a simple linear layer, that takes the embedding of the first encoder as input, where we use average-pooling over the sensors, such that we obtain a single embedding of 248 dimension per scan. The average pooling aims to counteracts the effect of the curse of dimensionality, and, while the PCA and t-sne embeddings differ slightly compared to the non-aggregated vectors, we observe better reconstruction and separation loss on the average-pooled vectors. We train the autoencoder for 10 epochs with a learning rate of 0.01, and use the following loss:

$$\mathcal{L} = 0.1 \cdot \mathcal{L}_{MSE} + \mathcal{L}_{\cosine} \quad (1)$$

with

$$\mathcal{L}_{\cosine}(\hat{x}_1, \hat{x}_2, y_1, y_2) = \begin{cases} 1 - \cos(\hat{x}_1, \hat{x}_2) & \text{if } y_1 = y_2 \\ \max\{0, \cos(\hat{x}_1, \hat{x}_2)\} & \text{otherwise} \end{cases} \quad (2)$$

To further emphasize the separation between the classes during the training, we scale the MSE loss with 10^{-1} , making the cosine similarity loss more important during training. After training, we observe that, while the training samples are very well separated (see Figure 11 in the Appendix), do not cluster well.

When looking at the embeddings of the test data with the train data (Figure 6), we observe that PCA is able to discriminate the different classes almost perfectly, but that the classification boundaries cannot be found with the train data alone.

Model	Accuracies (%)			
	Train	Test1	Test2	Test3
MLP	100.0	56.25	56.25	75.00
k -means ($k = 4$)	100.0	50.00	43.75	75.00
Logistic Regression	100.0	50.00	43.75	68.75

TABLE I

TEST ACCURACIES FOR $\mathcal{D}_{\text{Cross}}$.

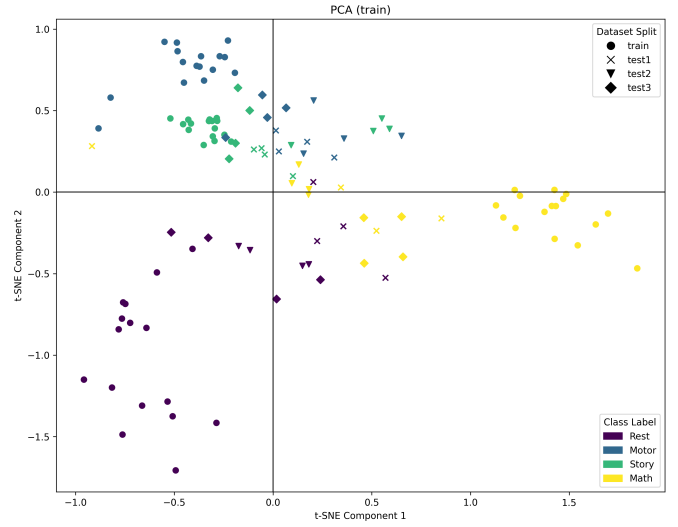


Fig. 6. The train and test1 data of $\mathcal{D}_{\text{Cross}}$.

V. DISCUSSION

VI. DISCUSSION & CONCLUSION

REFERENCES

APPENDIX

EXTRA FIGURES

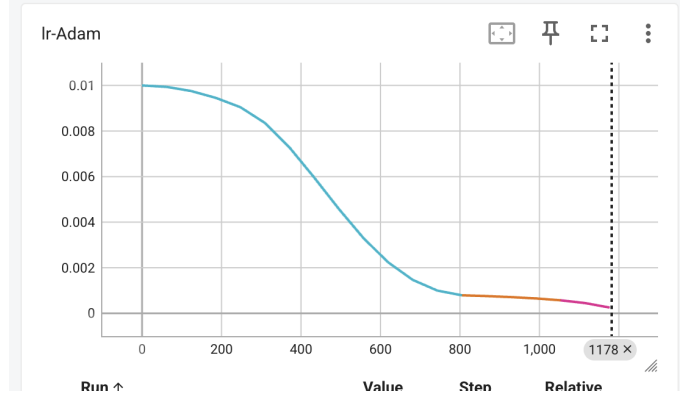


Fig. 7. The learning rate scheduler for the training of the Autoencoder.

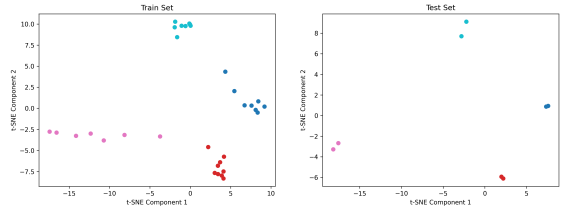


Fig. 8. The k-means embedded.

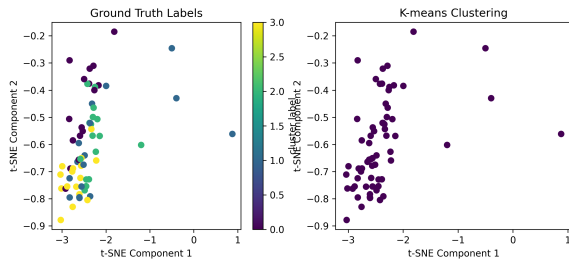


Fig. 9. The k-means embedded on $\mathcal{D}_{\text{Cross}}$ with k-means clusters fitted on $\mathcal{D}_{\text{Intra}}$. The important part of this plot are the x and y axis, which can be directly compared with the x and y axis from Figure 4, and therefore show the distribution shift between $\mathcal{D}_{\text{Intra}}$ and $\mathcal{D}_{\text{Cross}}$.

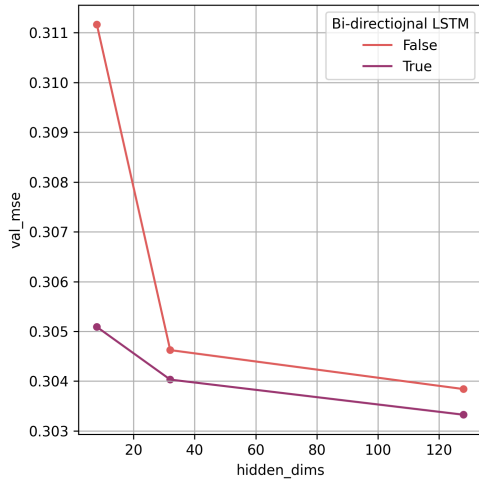


Fig. 10. Validation loss of different LSTM configuration for $\mathcal{D}_{\text{Cross}}$.

AI STATEMENT

No AI was used for any part of this project, including research, code generation, and text generation.

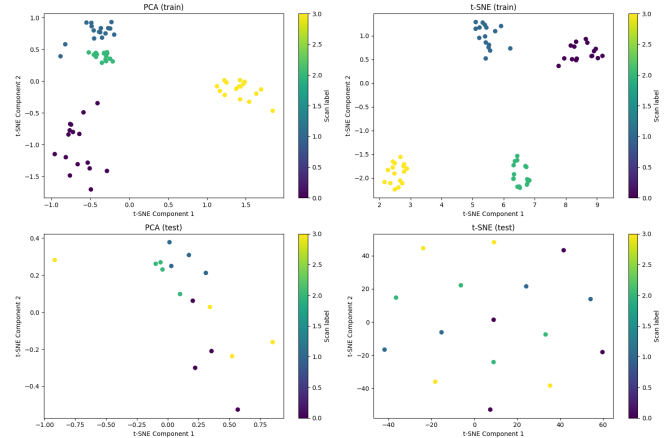


Fig. 11. Caption